

AI ASSISTED CODING

LAB-I

ASSIGNMENT-1

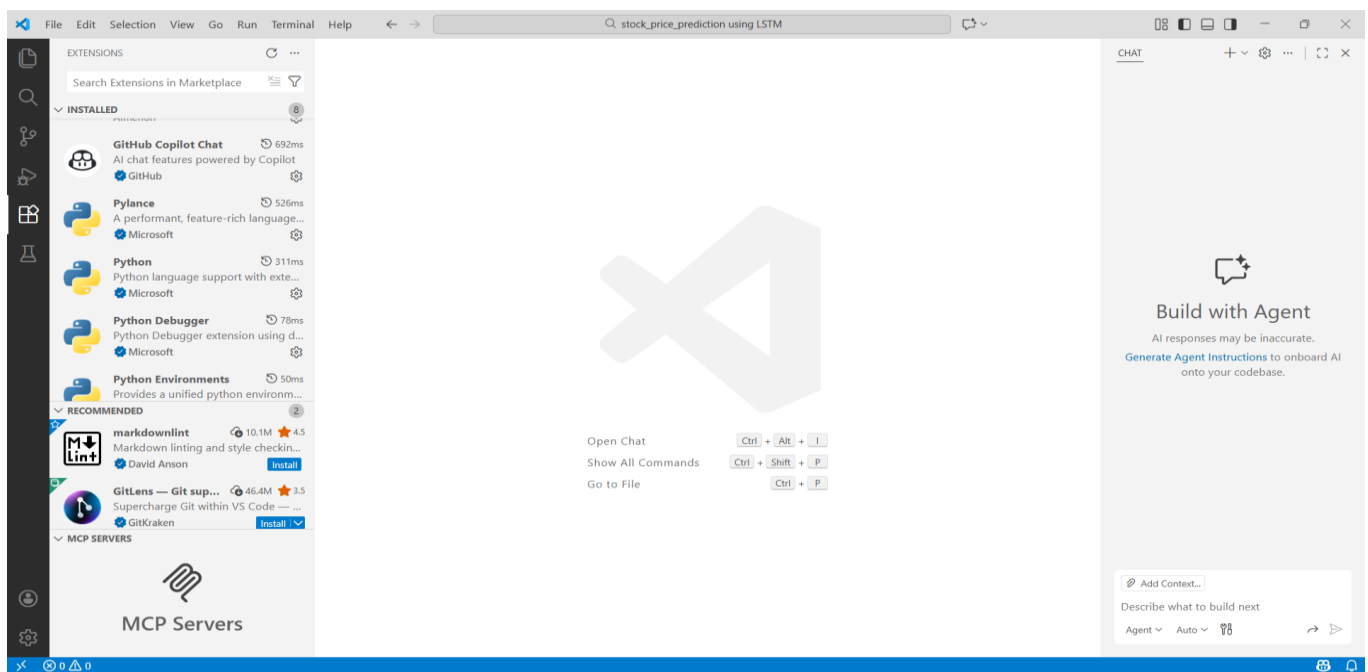
NAME:N.SATWIKA

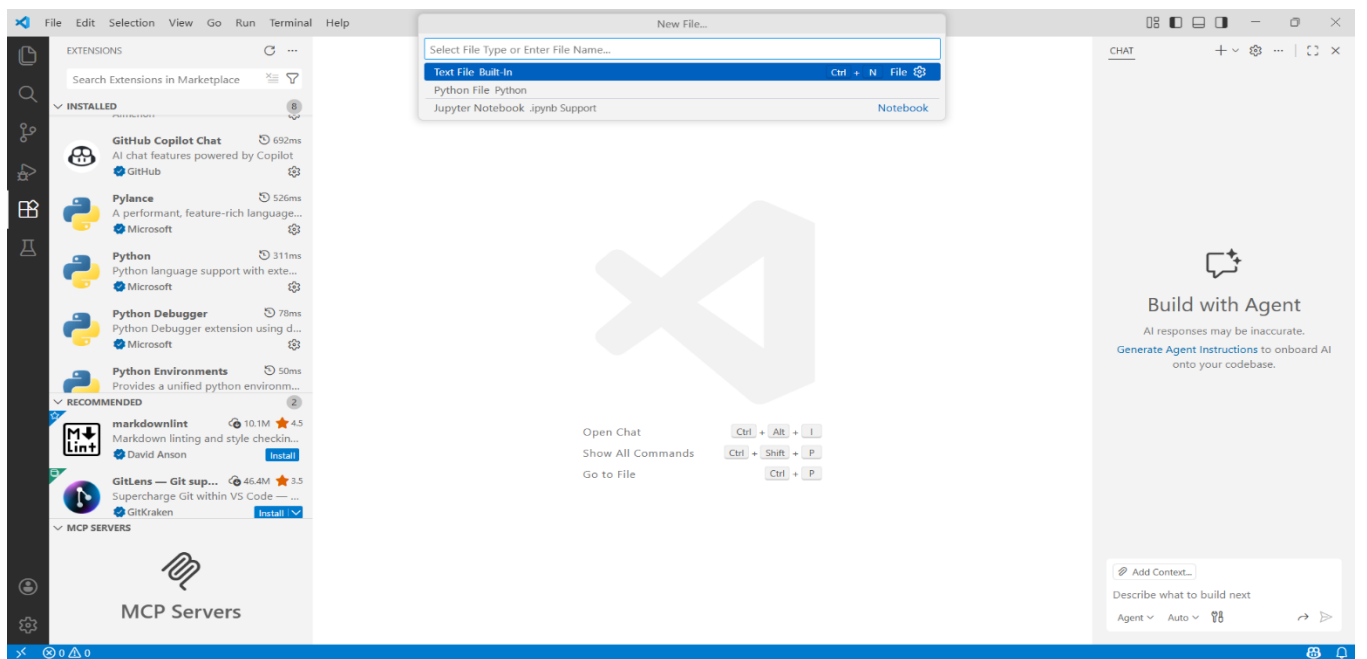
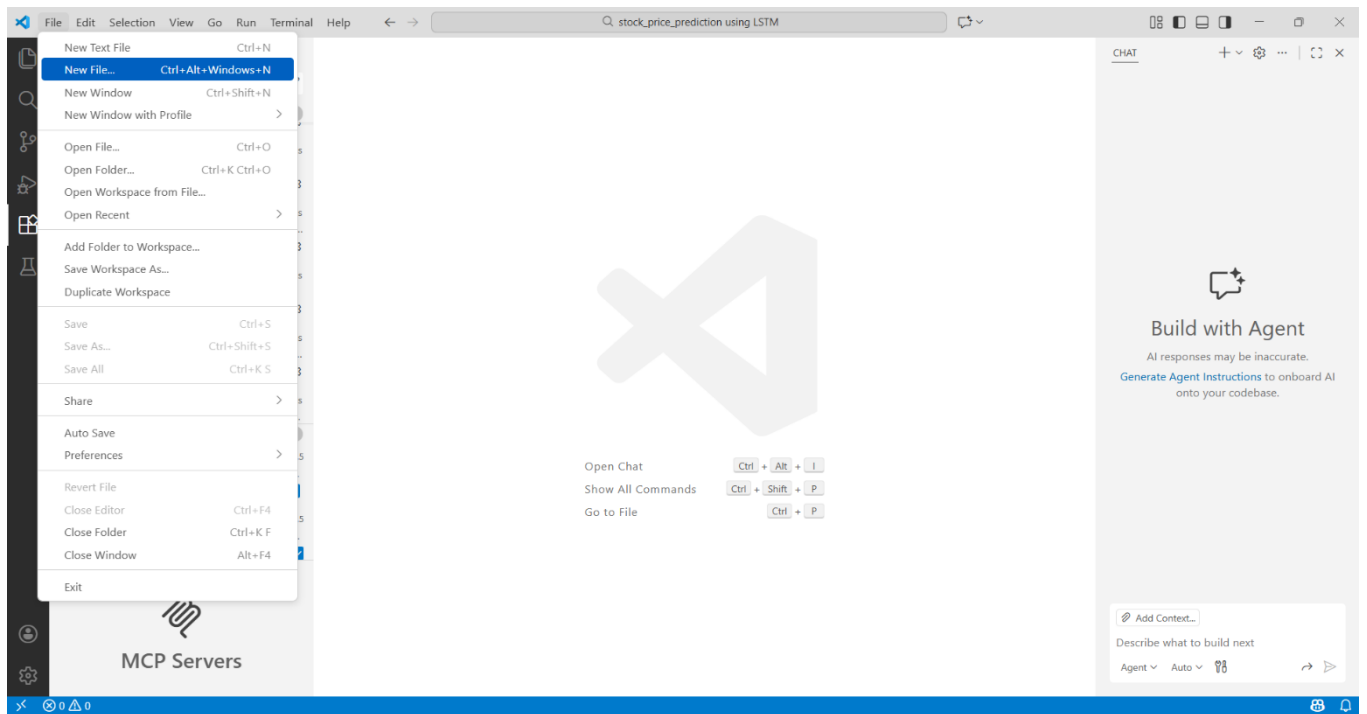
BATCH:28

ROLL NO.:2303A51690

TASK 0:

- Install and configure GitHub Copilot in VS Code.





TASK 1: AI-Generated Logic Without Modularization (Fibonacci Sequence Without Functions).

PROMPT: Write a python program to Fibonacci without functions.

CODE AND OUPUT:

```
1 #write a python program to fibonacci sequence without functions.
2 n = int(input("Enter the number of terms in Fibonacci sequence: "))
3 a, b = 0, 1
4 count = 0
5 if n <= 0:
6     print("Please enter a positive integer.")
7
8 elif n == 1:
9     print("Fibonacci sequence up to", n, ":")
10    print(a)
11 else:
12     print("Fibonacci sequence:")
13     while count < n:
14         print(a, end=" ")
15         nth = a + b
16         a = b
17         b = nth
18         count += 1
```

PS C:\Users\HP\OneDrive\Desktop\stock_price_prediction using LSTM> C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe "C:\Users\HP\OneDrive\Desktop\AI ASSISTED CODING\AI (Assignment-1.3).py"

Enter the number of terms in Fibonacci sequence: 9

Fibonacci sequence:

0 1 1 2 3 5 8 13 21

PS C:\Users\HP\OneDrive\Desktop\stock_price_prediction using LSTM> █

```
1 #write a python program to fibonacci sequence without functions.
2 n = int(input("Enter the number of terms in Fibonacci sequence: "))
3 a, b = 0, 1
4 count = 0
5 if n <= 0:
6     print("Please enter a positive integer.")
7
8 elif n == 1:
9     print("Fibonacci sequence up to", n, ":")
10    print(a)
11 else:
12     print("Fibonacci sequence:")
13     while count < n:
14         print(a, end=" ")
15         c = a + b
16         a = b
17         b = c
18         count += 1
```

PS C:\Users\HP\OneDrive\Desktop\stock_price_prediction using LSTM> C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe "C:\Users\HP\OneDrive\Desktop\AI ASSISTED CODING\AI (Assignment-1.3).py"

Enter the number of terms in Fibonacci sequence: 9

Fibonacci sequence:

0 1 1 2 3 5 8 13 21

PS C:\Users\HP\OneDrive\Desktop\stock_price_prediction using LSTM> █

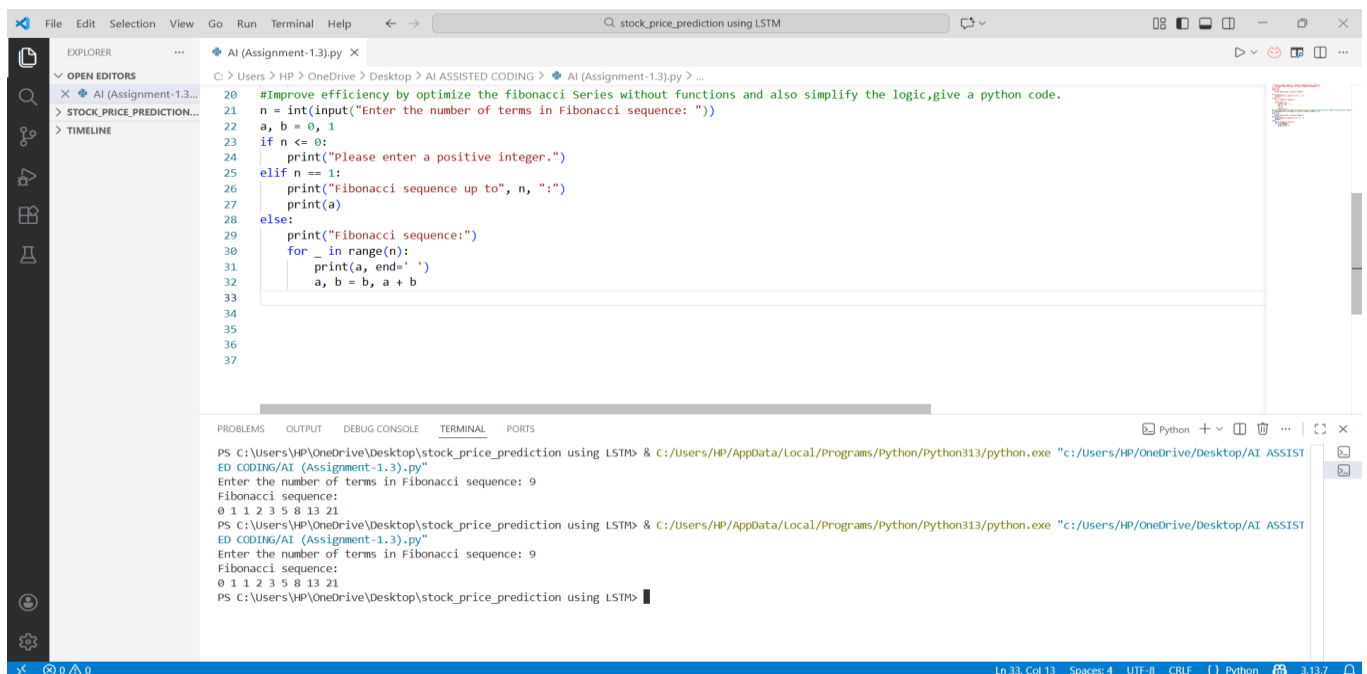
JUSTIFICATION:

1. All Fibonacci logic is written in a single block of code.
2. Easy for beginners to understand basic sequence generation.
3. Code readability decreases as the program grows.
4. Logic cannot be reused in other programs.
5. Suitable only for small and simple applications.

TASK:2:- AI Code Optimization & Cleanup (Improving Efficiency).

PROMPT: Improve efficiency by optimize the fibonacci sequence without using functions and also simplify the logic,give a python code.

CODE AND OUTPUT:



The screenshot shows a Visual Studio Code editor window with a Python file named 'AI (Assignment-1.3).py'. The code is a script that prompts the user for the number of terms in a Fibonacci sequence and then prints the sequence. The code is optimized to avoid using functions and to simplify the logic. The terminal output shows the execution of the script, where the user enters '9' and the program outputs the first 9 terms of the Fibonacci sequence: '0 1 1 2 3 5 8 13 21'.

```
20 #Improve efficiency by optimize the fibonacci Series without functions and also simplify the logic,give a python code.
21 n = int(input("Enter the number of terms in Fibonacci sequence: "))
22 a, b = 0, 1
23 if n <= 0:
24     print("Please enter a positive integer.")
25 elif n == 1:
26     print("Fibonacci sequence up to", n, ":")
27     print(a)
28 else:
29     print("Fibonacci sequence:")
30     for _ in range(n):
31         print(a, end=' ')
32         a, b = b, a + b
33
34
35
36
37
```

Terminal Output:

```
PS C:\Users\HP\OneDrive\Desktop\stock_price_prediction using LSTM> & C:/Users/HP/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/HP/OneDrive/Desktop/AI ASSIST
ED CODING/AI (Assignment-1.3).py"
Enter the number of terms in Fibonacci sequence: 9
Fibonacci sequence:
0 1 1 2 3 5 8 13 21
PS C:\Users\HP\OneDrive\Desktop\stock_price_prediction using LSTM> & C:/Users/HP/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/HP/OneDrive/Desktop/AI ASSIST
ED CODING/AI (Assignment-1.3).py"
Enter the number of terms in Fibonacci sequence: 9
Fibonacci sequence:
0 1 1 2 3 5 8 13 21
PS C:\Users\HP\OneDrive\Desktop\stock_price_prediction using LSTM>
```

JUSTIFICATION:

1. Optimized logic reduces unnecessary calculations.
2. Code becomes simpler and easier to read.
3. Execution efficiency is improved without using functions.
4. Shows how AI can enhance performance through optimization.
5. Still limited in scalability due to lack of modularization.

TASK 3: Modular Design Using AI Assistance (Fibonacci Using Functions).

PROMPT: Write a python program to fibonacci sequence using functions and give a code with comments.

CODE AND OUTPUT:

The screenshot shows a Visual Studio Code editor with a Python file named `AI (Assignment-1.3).py`. The code defines a `fibonacci_sequence(n)` function that generates a Fibonacci sequence up to `n` terms. The function uses a loop to calculate the sequence and returns it. The main part of the script prompts the user to enter the number of terms, validates the input, and prints the resulting sequence as a space-separated string.

The terminal window at the bottom shows the command prompt output for the command `python stock_price_prediction using LSTM`. The output indicates that the script is running in the directory `C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe` and prompts the user to enter the number of terms in the Fibonacci sequence. The user has entered `9`, and the output shows the first 9 terms of the Fibonacci sequence: `0 1 1 2 3 5 8 13 21`.

```

33 #write a python program to fibonacci sequence using functions and give a code with comments.
34 def fibonacci_sequence(n):
35     """Generate Fibonacci sequence up to n terms."""
36     a, b = 0, 1 # Initialize the first two Fibonacci numbers
37     sequence = [] # List to store the Fibonacci sequence
38
39     for _ in range(n):
40         sequence.append(a) # Append the current number to the sequence
41         a, b = b, a + b # Update a and b to the next two numbers in the sequence
42
43     return sequence # Return the complete Fibonacci sequence
44 # Get user input
45 n = int(input("Enter the number of terms in Fibonacci sequence: "))
46 if n <= 0:
47     print("Please enter a positive integer.")
48 else:
49     fib_sequence = fibonacci_sequence(n) # Call the function to get the Fibonacci sequence
50     print("Fibonacci sequence:")
51     print(' '.join(map(str, fib_sequence))) # Print the sequence as a space-separated string
52
53

```

```

PS C:\Users\HP\OneDrive\Desktop>stock_price_prediction using LSTM & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/HP/OneDrive/Desktop/AI ASSIST
ED CODING/AI (Assignment-1.3).py"
Enter the number of terms in Fibonacci sequence: 9
Fibonacci sequence:
0 1 1 2 3 5 8 13 21
PS C:\Users\HP\OneDrive\Desktop>stock_price_prediction using LSTM

```

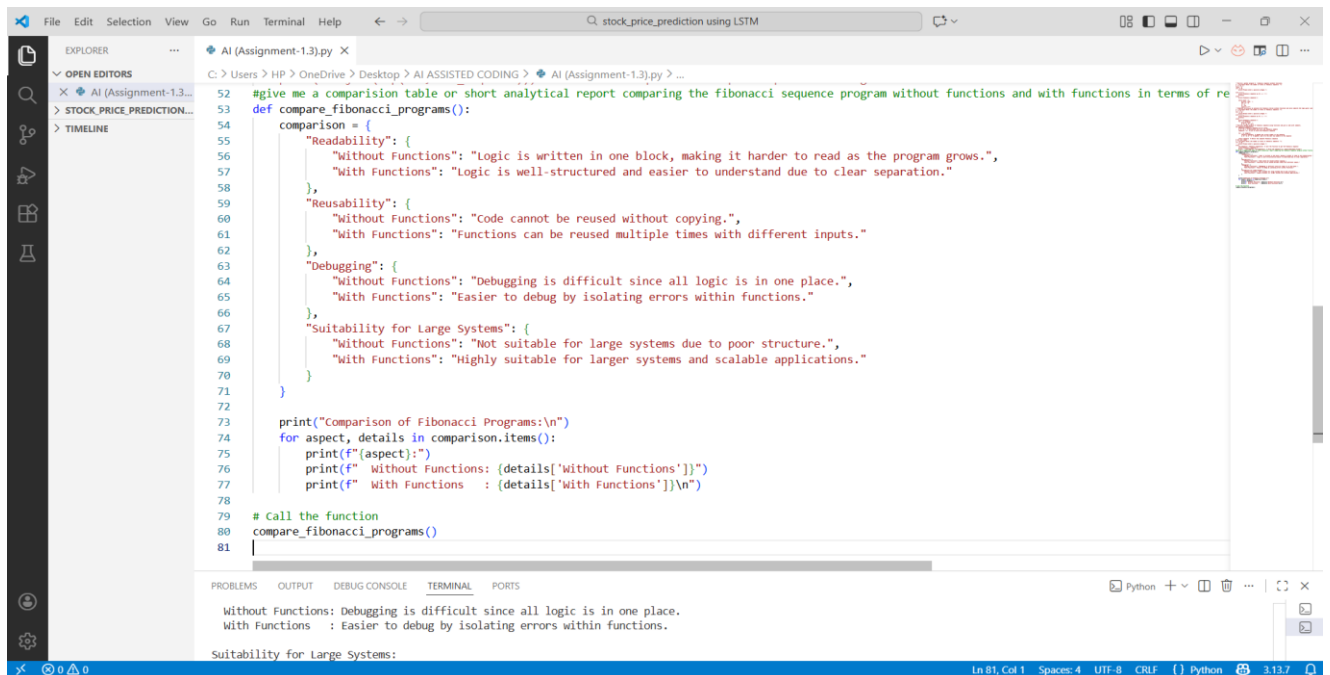
JUSTIFICATION:

1. Fibonacci logic is separated into a function.
2. Improves code readability and structure.
3. Enables reusability of the Fibonacci function.
4. Makes debugging easier by isolating errors.
5. Suitable for larger and real-world software systems.

TASK 4: Comparative Analysis – Procedural vs Modular Fibonacci Code.

PROMPT: give me a comparison table or short analytical report comparing the fibonacci sequence program without functions and with functions in terms of readability, reusability, and efficiency,give a code in python .

CODE AND OUTPUT:



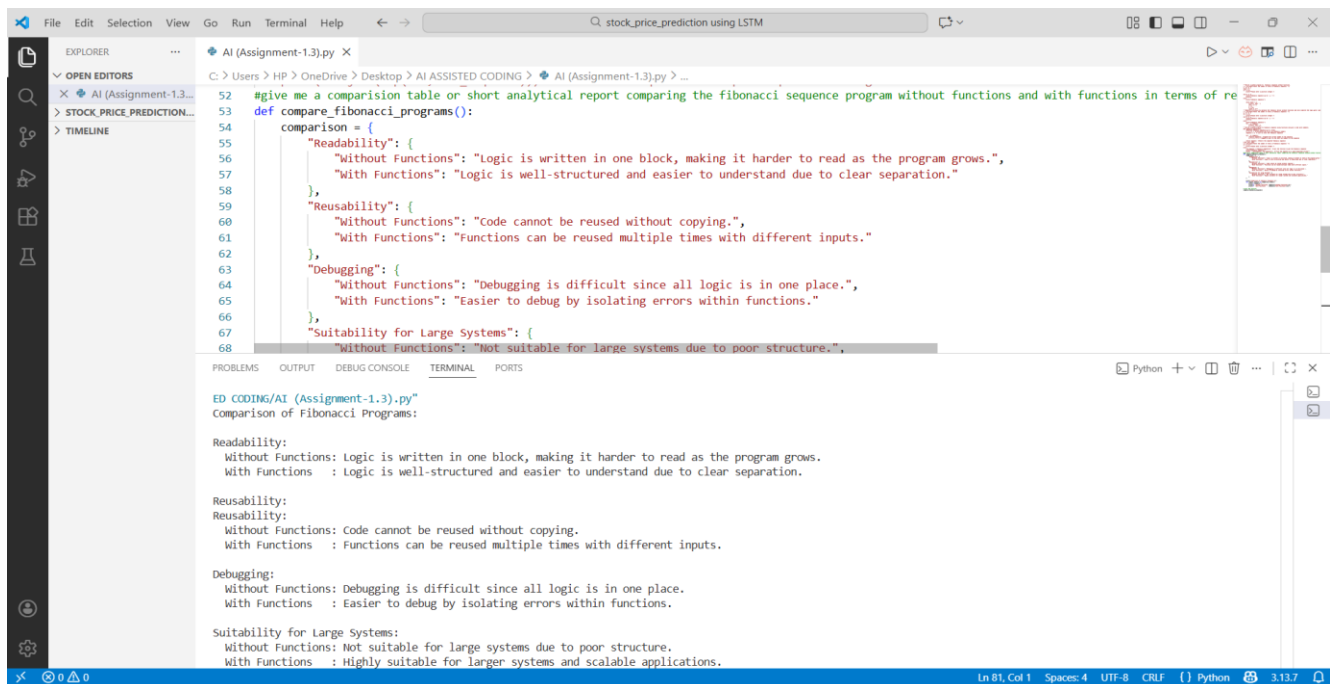
The screenshot shows the VS Code editor with the file 'AI (Assignment-1.3).py' open. The code defines a function 'compare_fibonacci_programs()' that returns a dictionary 'comparison' with four keys: 'Readability', 'Reusability', 'Debugging', and 'Suitability for Large Systems'. Each key has a list of two strings comparing procedural and modular code. The function is then called and the results are printed in a formatted way.

```
52 #give me a comparison table or short analytical report comparing the fibonacci sequence program without functions and with functions in terms of re
53 def compare_fibonacci_programs():
54     comparison = {
55         "Readability": [
56             "Without Functions": "Logic is written in one block, making it harder to read as the program grows.",
57             "With Functions": "Logic is well-structured and easier to understand due to clear separation."
58         ],
59         "Reusability": [
60             "Without Functions": "Code cannot be reused without copying.",
61             "With Functions": "Functions can be reused multiple times with different inputs."
62         ],
63         "Debugging": [
64             "Without Functions": "Debugging is difficult since all logic is in one place.",
65             "With Functions": "Easier to debug by isolating errors within functions."
66         ],
67         "Suitability for Large Systems": [
68             "Without Functions": "Not suitable for large systems due to poor structure.",
69             "With Functions": "Highly suitable for larger systems and scalable applications."
70         ]
71     }
72
73     print("Comparison of Fibonacci Programs:\n")
74     for aspect, details in comparison.items():
75         print(f"({aspect}):")
76         print(f"Without Functions: {details['Without Functions']}")
77         print(f"With Functions: {details['With Functions']}\n")
78
79 # Call the function
80 compare_fibonacci_programs()
81
```

The terminal output shows the results of the function call:

```
Without Functions: Debugging is difficult since all logic is in one place.
With Functions: Easier to debug by isolating errors within functions.

Suitability for Large Systems:
```



This screenshot shows the same VS Code editor, but the terminal window is now displaying the formatted output of the 'compare_fibonacci_programs()' function. The output is organized into sections for each aspect: Readability, Reusability, Debugging, and Suitability for Large Systems. Each section shows the comparison between 'Without Functions' and 'With Functions'.

```
ED CODING/AI (Assignment-1.3).py"
Comparison of Fibonacci Programs:

Readability:
Without Functions: Logic is written in one block, making it harder to read as the program grows.
With Functions: Logic is well-structured and easier to understand due to clear separation.

Reusability:
Without Functions: Code cannot be reused without copying.
With Functions: Functions can be reused multiple times with different inputs.

Debugging:
Without Functions: Debugging is difficult since all logic is in one place.
With Functions: Easier to debug by isolating errors within functions.

Suitability for Large Systems:
Without Functions: Not suitable for large systems due to poor structure.
With Functions: Highly suitable for larger systems and scalable applications.
```

JUSTIFICATION:

Procedural vs Modular Fibonacci Programs

1. Code Clarity:

Procedural code is harder to read as all logic is written together, while modular code is clearer due to separation into functions.

2. Reusability:

Procedural code cannot be reused easily, whereas modular code allows the Fibonacci function to be reused multiple times.

3. Debugging Ease:

Debugging is difficult in procedural code, but modular code makes error identification easier by isolating logic.

4. Scalability:

Procedural code is not suitable for large programs, while modular code supports growth and scalability.

5. Maintainability:

Changes in procedural code affect the entire program, whereas modular code allows easy updates within functions.

6. Professional Practice:

Modular programming follows standard software development practices and is preferred in real-world systems.

TASK 5: AI-Generated Iterative vs Recursive Fibonacci Approaches (Different Algorithmic Approaches for Fibonacci Series).

PROMPT: Generate iterative and recursive implementations of the Fibonacci series. Explain the execution flow of each approach and compare them based on time complexity, space complexity, performance for large n, and cases where recursion should be avoided.give a python code.

CODE AND OUTPUT:

The screenshot shows a VS Code editor window with the file 'AI (Assignment-1.3).py' open. The code implements two functions: `fibonacci_iterative(n)` and `fibonacci_recursive(n, sequence=None)`. The iterative function uses a loop to calculate the sequence, while the recursive function uses function calls. Comments explain the execution flow of each approach and compare them based on time and space complexity. The terminal output shows the sequence of numbers generated by both functions for n=14.

```
81 # Generate iterative and recursive implementations of the Fibonacci series. Explain the execution flow of each approach and compare them based on ti
82 def fibonacci_iterative(n):
83     # Generate Fibonacci sequence up to n terms using an iterative approach.
84     a, b = 0, 1
85     sequence = []
86     for _ in range(n):
87         sequence.append(a)
88         a, b = b, a + b
89     return sequence
90 def fibonacci_recursive(n, sequence=None):
91     # Generate Fibonacci sequence up to n terms using a recursive approach.
92     if sequence is None:
93         sequence = []
94     if n <= 0:
95         return sequence
96     if len(sequence) == 0:
97         sequence.append(0)
98     elif len(sequence) == 1:
99         sequence.append(1)
100    else:
101        next_value = sequence[-1] + sequence[-2]
102        sequence.append(next_value)
103    return fibonacci_recursive(n - 1, sequence)
104 # Get user input
105 n = int(input("Enter the number of terms in Fibonacci sequence: "))
106 if n <= 0:
107     print("Please enter a positive integer.")
108 else:
109     print("Fibonacci sequence (Iterative):")
110     print(' '.join(map(str, fibonacci_iterative(n))))
111     print("Fibonacci sequence (Recursive):")
112     print(' '.join(map(str, fibonacci_recursive(n))))
```

Terminal Output:

```
0 1 1 2 3 5 8 13 21 34 55 89 144 233
PS C:\Users\HP\OneDrive\Desktop\stock_price_prediction using LSTM>
```

The screenshot shows the same VS Code editor window, but with an additional function `compare_fibonacci_programs()` added at the top. This function calls both `fibonacci_iterative(n)` and `fibonacci_recursive(n)` for a given `n` and prints the results. The terminal output shows the execution of this function for `n=14`, displaying the Fibonacci sequence generated by both methods.

```
80 compare_fibonacci_programs()"""
81 # Generate iterative and recursive implementations of the Fibonacci series. Explain the execution flow of each approach and compare them based on ti
82 def fibonacci_iterative(n):
83     # Generate Fibonacci sequence up to n terms using an iterative approach.
84     a, b = 0, 1
85     sequence = []
86     for _ in range(n):
87         sequence.append(a)
88         a, b = b, a + b
89     return sequence
90 def fibonacci_recursive(n, sequence=None):
91     # Generate Fibonacci sequence up to n terms using a recursive approach.
92     if sequence is None:
93         sequence = []
94     if n <= 0:
95         return sequence
96     if len(sequence) == 0:
97         sequence.append(0)
98     elif len(sequence) == 1:
99         sequence.append(1)
100    else:
101        next_value = sequence[-1] + sequence[-2]
102        sequence.append(next_value)
103    return fibonacci_recursive(n - 1, sequence)
104 # Get user input
105 n = int(input("Enter the number of terms in Fibonacci sequence: "))
106 if n <= 0:
107     print("Please enter a positive integer.")
```

Terminal Output:

```
ED CODING/AI (Assignment-1.3).py"
Enter the number of terms in Fibonacci sequence: 14
Fibonacci sequence (Iterative):
0 1 1 2 3 5 8 13 21 34 55 89 144 233
Fibonacci sequence (Recursive):
0 1 1 2 3 5 8 13 21 34 55 89 144 233
PS C:\Users\HP\OneDrive\Desktop\stock_price_prediction using LSTM>
```

JUSTIFICATION:

1. Iterative approach uses loops and is memory efficient.
2. Recursive approach uses function calls and consumes more memory.
3. Recursive Fibonacci has higher time complexity due to repeated calls.

4. Iterative approach performs better for large values of n .
5. Recursion should be avoided in performance-critical applications.
6. Demonstrates different algorithmic paradigms clearly.

